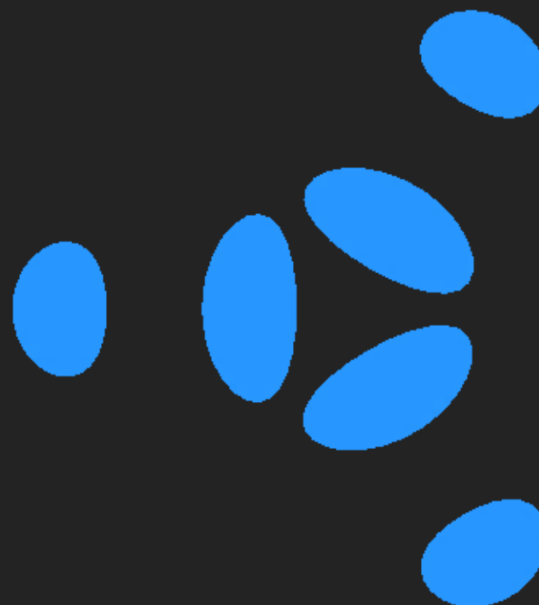




***COMEÇAREMOS
EM BREVE!***





especialização

RAN Intelligent Controller (RIC)

Prof. Dr. Jonas Augusto Kunzler

Open RAN:

Redes Abertas e Tecnologias Emergentes



Encontro de Seminários — 08:00–13:00

Projeto 1 — Apresentações (08:00–11:00) + Interface A1 e políticas

Disciplina 7: RAN Intelligent Controller (RIC)

Aula 03: Projeto 1 (40%) e Interface A1

Data: 13/06/2026 | Horário: 08:00–13:00 | Duração: 5h

08:00–11:00: Projeto 1 — apresentações (20 min); Polese + NGO §6.1 (sorteio) · correlacionar **Módulo 5** (A1/O1/E2/O2)

11:00–13:00: retomada A1 (Módulo 5) · NGO Non-RT/rApps · lab srsRAN/ZMQ





Calendário — destaque Aula 03

Data / Horário	Conteúdos
09/06 18:30–22:00	Aula 01: Retomada O-RAN + Lab Open5GS/UERANSIM
11/06 18:30–22:00	Aula 02: Artigo NGO — OSS RIC e casos de uso
13/06 08:00–13:00	Aula 03: Projeto 1 + A1 + discussão srsRAN/ZMQ
16/06 18:30–22:00	Aula 04: OAI+FlexRIC — início Projeto 2
18/06 18:30–22:00	Aula 05: Design de xApps e SMO
20/06 08:00–13:00	Aula 06: Projeto 2 (40%) + validação

Acordos Didáticos

- **Projeto 1:** bloco 08:00–11:00; 20 min/grupo.
- **Presença:** início, três intervalos de 15 min e encerramento.
- **Participação:** ativa nas contribuições de discussão aos colegas.

Cronograma Previsto — Aula 03

- 08:00–11:00 — **Projeto 1**: apresentações (20 min/grupo, ordem sorteada)
- 11:00–11:30 — Retomada **Módulo 5**: A1, políticas, EI; A1 vs O1/E2
- 11:30–11:45 — NGO §3.1–§4: Non-RT OSC, pipeline rApps SUTD
- 11:45–12:00 — Intervalo (15 min)
- 12:00–12:45 — Discussão: **3 projetos Open5GS** · desagregação · srsRAN/ZMQ
- 12:45–13:00 — Síntese · encaminhamento Projeto 2 (OAI+FlexRIC)

Objetivos da Aula 03

Objetivo central

Apresentar Projeto 1 (Polese + NGO §6.1) correlacionando interfaces do Módulo 5

Ao final da aula

Consolidar A1/políticas/EI (Módulo 5) no fluxo NGO SUTD; discutir srsRAN/ZMQ; encaminhar lab OAI (Projeto 2)

Ponte — Aula 02 + Módulo 5 → Projeto 1

O seminário deve cruzar Polese (interfaces) com NGO (implementação SUTD)

- **Aula 02:** retomada Módulo 5 + survey OSS + casos ES/IM/PM e proxy E2.
- **Módulo 5 (Aula 03):** A1 (políticas/EI), O1 (FCAPS/PM), E2 (near-RT) — base teórica.
- **Projeto 1:** aprofundar **um** ponto §6.1 e dizer *qual interface* sustenta cada etapa.

Projeto 1 (40%) — enunciado

Leitura Polese + NGO · estudo de caso §6.1

- **Artigos:** Polese (Understanding O-RAN) + NGO (RIC Open Source Implementation).
- **Tema:** um dos 7 casos §6.1 — atribuído por **sorteio** (grupos com 4 colegas).
- **Apresentação:** 20 min por grupo (08:00–11:00); mesma ordem no Projeto 2.

Temas de sorteio — NGO §6.1 (ordem do artigo)

#	Tema	Foco do estudo de caso
1	Coleta de KPM	VES Collector → InfluxDB; KPMs de UEs na RAN
2	Localization-rApp	Métricas de rádio → andar (classificador)
3	UE-TP-rApp	Throughput UE; RSSI, RSRP, CQI, PRB, histórico
4	Cell-TP-rApp	Demanda por célula; saídas Localization + UE-TP
5	ES-rApp	On/off O-RU; demanda prevista (~65% simulado)
6	IM-rApp	Zonas de interferência; potência O-RU vizinho
7	PM-rApp	RUs ativos; RU malformado; feedback UE

KPM-xApp / TS-xApp complementam o fluxo, mas não aparecem aqui.

§6.1 × interfaces O-RAN (Módulo 5)

Tema §6.1	Entrada (dados)	Saída / ação
1 Coleta KPM	O1 / VES → InfluxDB	Base para todos os rApps
2 Localization	KPM (RSRP, SINR...)	EI-like → Cell-TP / ES
3 UE-TP	KPM + histórico	Previsão → Cell-TP
4 Cell-TP	Localization + UE-TP	Demanda por célula → ES/IM
5 ES-rApp	Cell-TP + O1	A1 (política) + O1 (sleep RRU)
6 IM-rApp	KPM + mapa interferência	O1 (potência O-RU)
7 PM-rApp	O1 + feedback UE	Alarme / manutenção preditiva

*TS-xApp recebe política **A1** do ES-rApp; KPM-xApp usa **E2**.*

Projeto 1 — formato e critérios

Bloco 08:00–11:00 · 20 min/grupo

- Citar **ambos** os artigos; aprofundar o **tema sorteado**.
- Correlacionar com **Módulo 5** (A1, E2, O1, O2).
- Analisar métricas, arquitetura e limitações — não só resumo dos slides.

Grupo 01 - Coleta de KPM

Grupo 02 - Localization-rApp

 Intervalo 01

Marcação de presença conforme acordos didáticos.

Grupo 03 - UE-TP-rApp

Grupo 04 - Cell-TP-rApp

Grupo 05 - ES-rApp

Intervalo 02

Marcação de presença conforme acordos didáticos.

Grupo 06 - IM-rApp

Grupo 07 - PM-rApp

 Intervalo 03

Marcação de presença conforme acordos didáticos.

 **Bloco docente — 11:00–11:45**

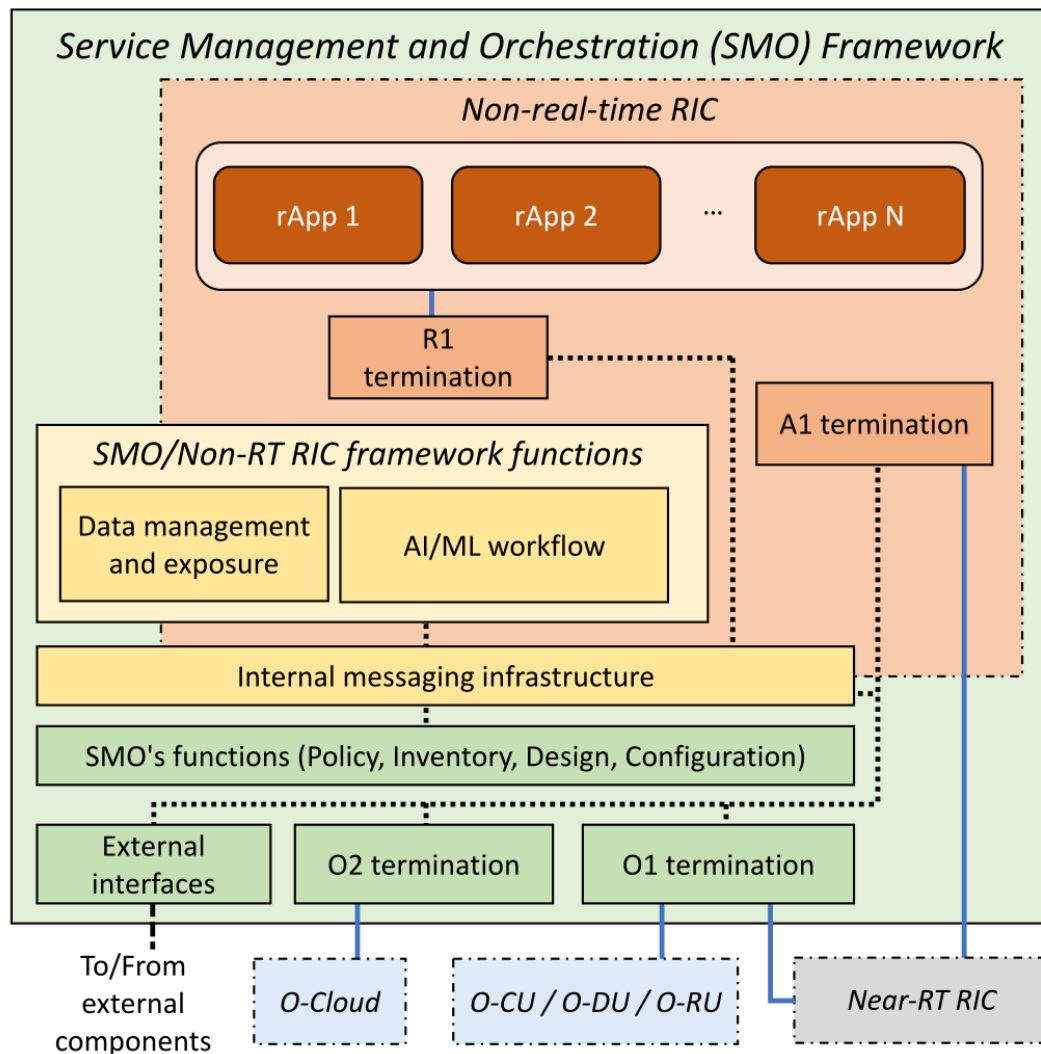
Retomada Módulo 5 (A1) · NGO Non-RT OSC

Non-RT RIC — papel no SMO

Orquestração lenta e orientada a dados

- Abriga **rApps** com visão de rede e serviços (políticas, previsão, planejamento).
- Pode operar com **intent-based management** e encadeamento com analytics/IA.
- Publica orientações para o Near-RT RIC pela interface **A1**.
- Tempo de ciclo: minutos a horas — adequado para ML (Localization, UE-TP, PM-rApp no SUTD).

Non-RT RIC — arquitetura funcional



- **Non-RT RIC:** controle >1 s; interface **A1**; hospeda rApps; políticas e ML de longo prazo.
- **SMO:** orquestra serviços, gerencia **O1/O2**, FCAPS, lifecycle, automação.
- **NGO §3.1:** ORAN-SC NONRTRIC — A1, O1, ICS/ DMaaP, rApp catalogue no SMO.

Interface A1 — visão geral (Polese)

Non-RT RIC ↔ Near-RT RIC

- **Propósito:** policy-based guidance; gestão de modelos ML; Enrichment Information (EI); intent de alto nível.
- **Características:** não em tempo real; ciclo >1 s; sintaxe baseada em JSON schema.
- **Protocolo:** A1AP sobre REST/HTTP; pares HTTP client/server por serviço.
- **Escopo:** políticas, ML models e EI podem referir-se a grupo de UEs ou UE específico.

Policy: estrutura e tipos

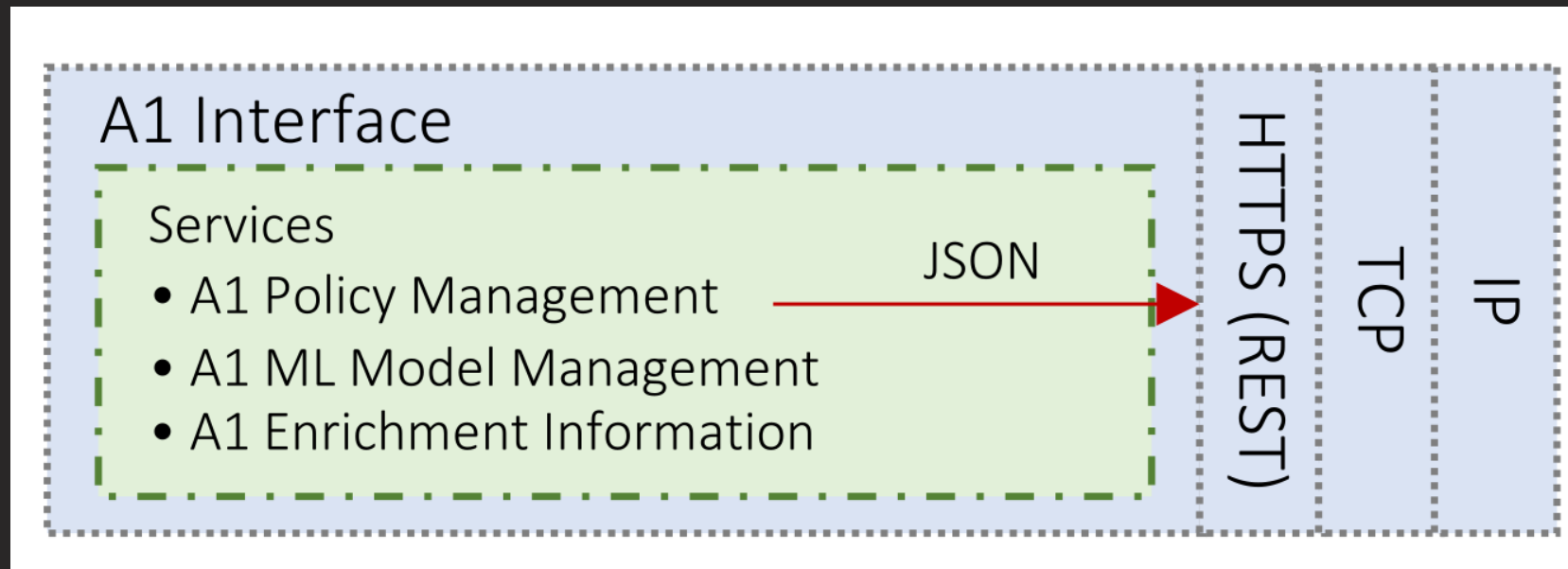
- **Estrutura JSON:** policy identifier, scope identifier, policy statements; schema por policy type.
- **Scope:** UE, grupo de UEs, slices, cells, bearers, application classes.
- **Policy statements:** policy resources (condições) e policy objectives (metas QoS/KPI).
- **Tipos:** throughput maximization, traffic steering, QoS targets, energy saving — intent via O1 reporting e feedback A1.
- **Life cycle:** APIs create/update/delete/query no Near-RT RIC.

A1 EI Service

- **Objetivo:** melhorar performance da RAN com informação indisponível localmente no nó.
- **Exemplos:** previsão de carga; analytics agregados; mapas de contexto externo.
- **Fluxo:** Non-RT RIC/SMO relay via A1 EI para xApps; ou instrução de acesso direto a fontes EI.

NGO: saídas de Localization-rApp e Cell-TP-rApp funcionam como EI interna para ES/IM-rApps.

A1 — políticas como contratos JSON



Policy ID, scope e statements — Módulo 5, Aula 03; ES-rApp publica limiares SLA e células candidatas (NGO §4.4).

A1 — exemplos de objetivos de política

Do intent ao comportamento desejado

- Traffic steering — preferências de célula/RAT (TS-xApp).
- Energy saving — janelas e limiares para sleep modes (ES-rApp).
- QoS / slice — alocação e priorização por SLA.
- Políticas refinadas com feedback de **reporting O1** e loop via Near-RT RIC.

Estilo de integração

- Serviços A1 expõem APIs HTTP com papéis de **cliente/servidor por função**.
- Facilita integração com catálogos de políticas e portais de orquestração no SMO.
- Mantém o Non-RT RIC desacoplado da implementação interna do Near-RT RIC (OSC, FlexRIC...).

A1 — ciclo de vida de policy

Operações típicas

- **Create / Update / Delete** de políticas em um Near-RT RIC alvo.
- **Query** do conjunto de políticas ativas e versões.
- **Coerência:** políticas concorrentes tratadas no RIC (mitigação de conflito entre xApps).

NGO §4.4: ES-rApp atualiza/revoga política A1 se SLA degradar após sleep O-RU.

A1 vs O1 — não confundir

Política vs provisionamento

- **A1**: contratos de comportamento e intent para Near-RT RIC/xApps.
- **O1**: FCAPS sobre Network Elements — config, fault, performance, sleep/wake de RRU.
- Um objetivo de rede pode exigir **ambos**: policy (A1) e comando de configuração (O1).

NGO: ES-rApp → A1 (TS-xApp desvia tráfego) + O1 (comando sleep RRU).

Camadas complementares

- E2: assinaturas, reports, insert/control com vínculo próximo ao ciclo de RRM.
- A1: ajusta regras e modelos que os xApps aplicarão ao longo do tempo.
- Falha comum: tratar E2 como substituto de política A1 — são complementares.

NGO §5: KPM-xApp (E2) mede; ES-rApp (A1) orienta TS-xApp antes de ação O1.

Evolução da especificação

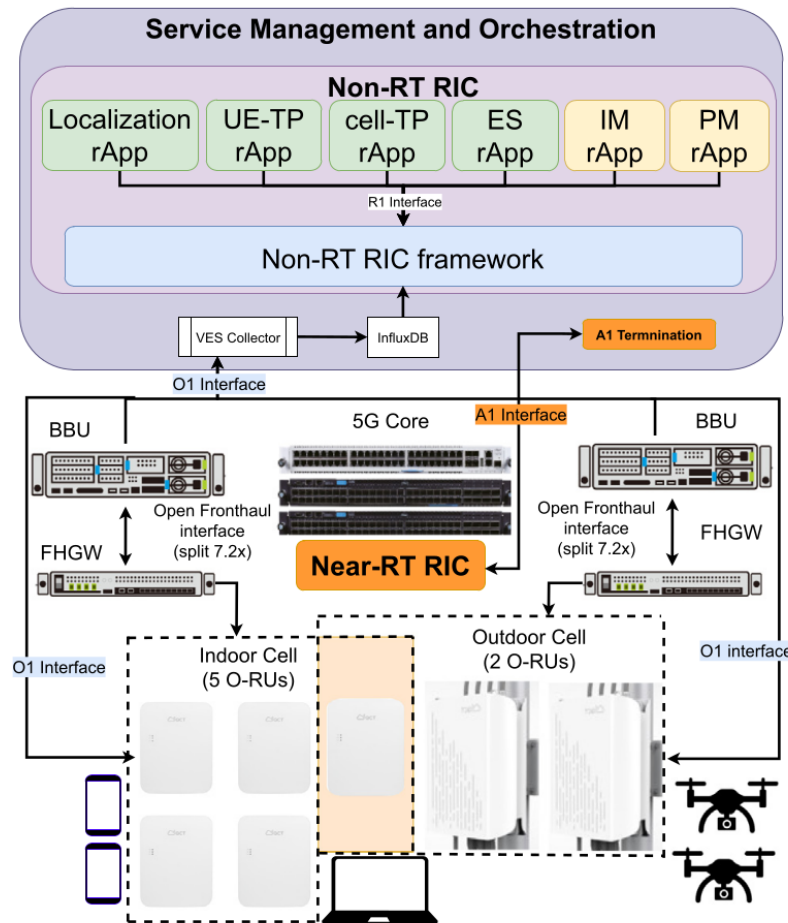
- Políticas podem referenciar modelos e artefatos ML.
- **Model management** — área de estudo; implementações variam (ORAN-SC, ONAP...).
- Deploy de imagens/modelos também pode passar por **O1 file management**.

NGO §6.1: DNN/LSTM nos rApps; inferência $<30 \mu s$; acurácia $>90\%$ por tarefa.

Seis fases aplicadas ao SUTD

1. **Coleta:** KPM via VES/O1 → InfluxDB.
2. **Treino offline:** DNN/LSTM (Localization, UE-TP, PM...) — dataset FCCLab.
3. **Publicação:** rApp catalogue ORAN-SC; versionamento de modelos.
4. **Deploy:** rApps no Non-RT; xApps no Near-RT (K8s).
5. **Inferência:** $<30 \mu\text{s}$; acurácia $>90\%$.
6. **Operação contínua:** políticas A1 + feedback O1; monitorar drift.

NGO Fig. 6 — arquitetura rApps + rede 5G



- KPM via VES Collector → InfluxDB → rApps ML.
- A1 liga Non-RT ao Near-RT.

Coleta de dados (pré-processamento)

- **UE:** Samsung S22 Ultra; walk test em corredores L4–L6 (building 2, SUTD).
- **Métricas:** RSRP, RSRQ, SINR, PDSCH PRBs/MCS, DL throughput (7 KPMs).
- **ML:** janela 10 amostras (5 s); split 70/30; normalização [0,1].
- Dataset público: [FCCLab/sutd_5g_dataset_2023](https://github.com/FCCLab/sutd_5g_dataset_2023)

rApps, intent e loop A1 (síntese)

- **Intent** — objetivo de alto nível.
- **rApp** (Non-RT) — traduz intent em políticas **A1** e analytics; dados via **O1**.
- **xApp** (Near-RT) — executa com **E2** dentro dos limites da política A1.
- **ES-rApp** → **A1** → **TS-xApp** (**E2**) → **O1 sleep** — loop completo.

 Intervalo 04

Marcação de presença conforme acordos didáticos.

Temas de sorteio — NGO §6.1 (ordem do artigo)

#	Tema	Foco do estudo de caso
1	Coleta de KPM	VES Collector → InfluxDB; KPMs de UEs na RAN
2	Localization-rApp	Métricas de rádio → andar (classificador)
3	UE-TP-rApp	Throughput UE; RSSI, RSRP, CQI, PRB, histórico
4	Cell-TP-rApp	Demanda por célula; saídas Localization + UE-TP
5	ES-rApp	On/off O-RU; demanda prevista (~65% simulado)
6	IM-rApp	Zonas de interferência; potência O-RU vizinho
7	PM-rApp	RUs ativos; RU malformado; feedback UE

KPM-xApp / TS-xApp complementam o fluxo, mas não aparecem aqui.

§6.1 × interfaces O-RAN (Módulo 5)

Tema §6.1	Entrada (dados)	Saída / ação
1 Coleta KPM	O1 / VES → InfluxDB	Base para todos os rApps
2 Localization	KPM (RSRP, SINR...)	EI-like → Cell-TP / ES
3 UE-TP	KPM + histórico	Previsão → Cell-TP
4 Cell-TP	Localization + UE-TP	Demanda por célula → ES/IM
5 ES-rApp	Cell-TP + O1	A1 (política) + O1 (sleep RRU)
6 IM-rApp	KPM + mapa interferência	O1 (potência O-RU)
7 PM-rApp	O1 + feedback UE	Alarme / manutenção preditiva

*TS-xApp recebe política **A1** do ES-rApp; KPM-xApp usa **E2**.*

Laboratório 3 projetos Open5GS · desagregação · srsRAN/ZMQ

Três projetos Open5GS — visão geral

Projeto	Diretório	Desagregação RAN	Aula
1	<code>open5gs-containerized</code>	UERANSIM: gNB+UE no mesmo container	01
2	<code>open5gs-container-srsRAN</code>	gNB e UE em containers separados + srsRAN ZMQ	02–03
3	<code>open5gs-container-cudu-srsRAN</code>	CU e DU em containers individuais (F1)	03+
4	<code>oai-cn-gnb</code>	OAI + FlexRIC (E2)	04–06

Escalada, o que muda em cada passo?

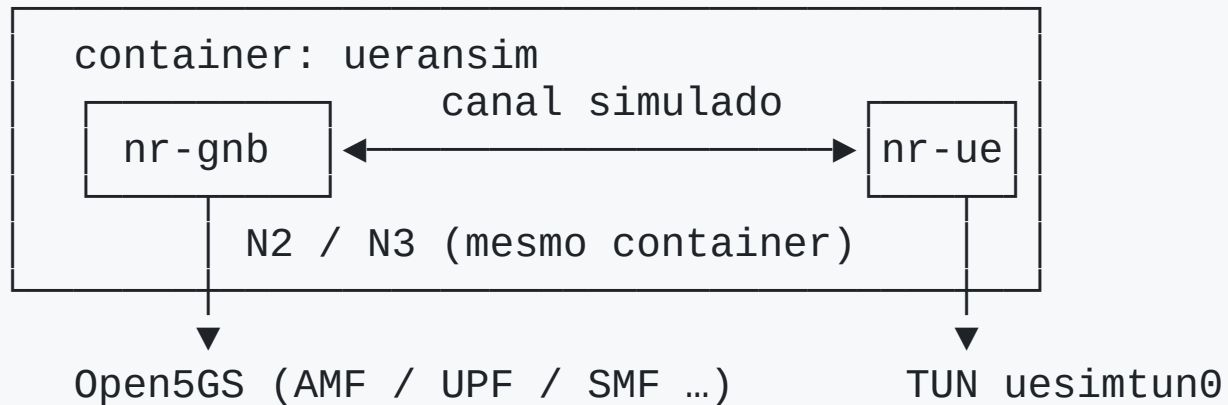
1. **Monólito UERANSIM** — um processo/container; canal rádio simulado internamente.
2. **gNB || UE** — funções em containers distintos; enlace `net-air` (UERANSIM) ou `net-zmq` (srsRAN).
3. **PHY real (ZMQ)** — srsRAN Project gNB + srsUE; amostras IQ via driver ZMQ.
4. **CU || DU** — `srscu` e `srsdu` separados; interfaces **F1-C** e **F1-U** (3GPP / O-RAN WG5).
5. **E2 (futuro)** — OAI + FlexRIC; KPM near-RT.

Projeto 1 — open5gs-containerized

Aula 01 · UERANSIM integrado (gNB + UE)

- **Compose RAN:** `ueransim/docker-compose.yaml` — container único `ueransim`.
- **Entrypoint:** inicia `nr-gnb` e `nr-ue` no **mesmo container** (`entrypoint.sh`).
- **Redes:** N2 `10.20.0.101` · N3 `10.30.0.11` — gNB e UE compartilham o namespace de rede do host Docker.
- **Objetivo:** validar 5GC (registro, PDU Session, N2/N3) sem desagregar RAN.
- **Limitação:** sem PHY real; canal gNB↔UE é abstração interna do simulador.

Projeto 1 — diagrama lógico



Subida: `up_core.sh` → `up_ran.sh`.

Projeto 2 — open5gs - container - srsRAN

Aulas 02-03 · dois saltos de desagregação

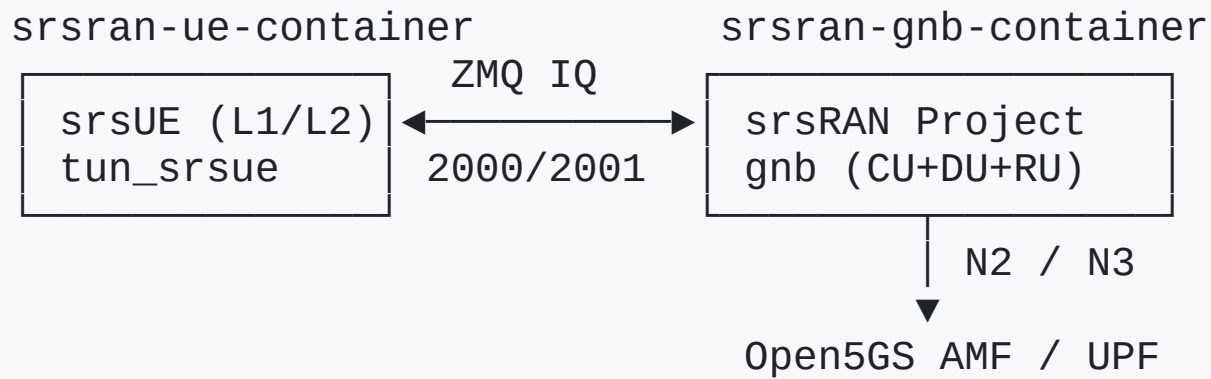
- **Passo A — UERANSIM split:** `ueransim-gnb` (N2/N3) + `ueransim-ue` — enlace **net-air** (`10.70.0.0/16`).
- **Passo B — srsRAN (perfil `srsran`):** `srsran-gnb` + `srsran-ue` — enlace **net-zmq** (`172.31.250.0/24`).
- **gNB:** binário `gnb` (srsRAN Project) — CU/DU agregados no **mesmo container**.
- **UE:** srsRAN 4G `srsue` (imagem local `srsue:latest`).
- **Ordem:** `up_core.sh` → `build-srsue.sh` → `up_ran.sh` (gNB ZMQ antes do UE).

Projeto 2 — containers e redes

Container	Função	Redes principais
ueransim-gnb	gNB simulado	N2 10.20.0.100 · N3 10.30.0.100 · air 10.70.0.100
ueransim-ue	UE simulado	air 10.70.0.200
srsran-gnb	gNB srsRAN (PHY real)	N2 10.20.0.101 · N3 10.30.0.101 · zmq 172.31.250.10
srsran-ue	srsUE (ZMQ)	zmq 172.31.250.20 · TUN tun_srsue

UERANSIM e srsRAN podem coexistir; perfil `--profile srsran` ativa a pilha com PHY.

Projeto 2 — diagrama srsRAN ZMQ



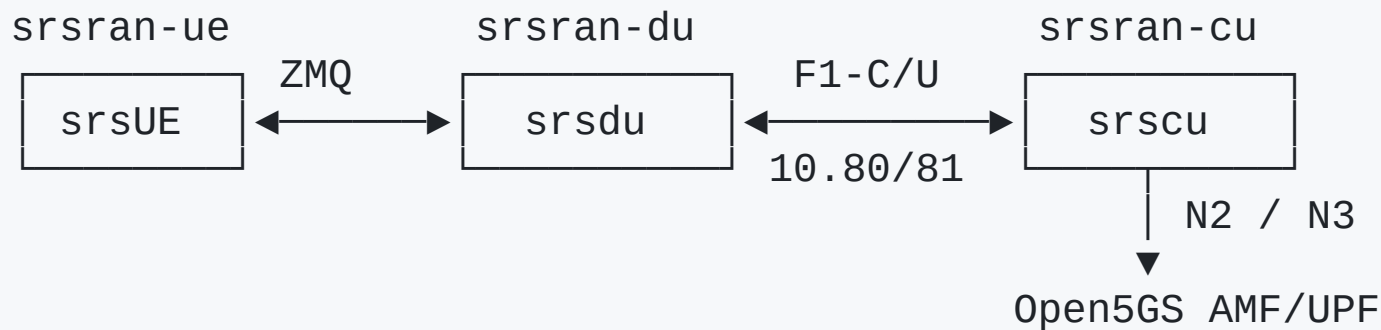
Projeto 3 — open5gs - container - cudu - srsRAN

Dois modos RAN no mesmo repositório

- Modo **e2e** (validado): `gnb_split_8` — CU e DU no mesmo processo, container `srsran-cu-du`; ZMQ com srsUE.
- Modo **strict-cudu** (experimental): `srscu` e `srsdu` em containers separados; F1 entre CU e DU; ZMQ no DU.
- Imagem CU/DU estrita: `build local open5gs-srsran-cudu-zmq:latest ( build-srsran-cudu-zmq.sh )`.

Nota: `aetherproject/srsran-gnb` serve ao **e2e**, mas *não* garante CU/DU independentes.

Projeto 3 — modo **strict-cudu** (CU || DU)



- **F1-C:** `net-f1c` — CU `10.80.0.10` ↔ DU `10.80.0.30`
- **F1-U:** `net-f1u` — CU `10.81.0.10` ↔ DU `10.81.0.30`
- **N2/N3:** apenas no CU — `10.20.0.110` / `10.30.0.110`
- **ZMQ:** **srsUE** aponta para `srsran-du` (`ue.conf`)

Projeto 3 — comandos e validação

Modo E2E (gnb_split_8)

- `./scripts/up.sh` OU `up_core.sh` + `up_ran.sh`
- `./scripts/test-srsue-e2e.sh` — TUN, PDU, ping via `tun_srsue`

Modo strict CU/DU

- `./scripts/build-srsran-cudu-zmq.sh` · `probe-srsran-image.sh`
- `./scripts/up_strict_cudu.sh`
- `SRSRAN_PROFILE=strict-cudu ./scripts/test-srsue-e2e.sh`

Comparativo — interfaces por projeto

Interface	Proj. 1	Proj. 2	Proj. 3 strict
N2 (NGAP)	✓ gNB	✓ srsRAN gNB	✓ CU
N3 (GTP-U)	✓ gNB	✓ srsRAN gNB	✓ CU-UP
gNB ↔ UE	interno	net-air / ZMQ	ZMQ (DU ↔ UE)
F1	—	— (agregado)	✓ CU ↔ DU
E2	—	—	—
PHY real	✗	✓ (ZMQ)	✓ (ZMQ no DU)

Software Radio Systems — srsRAN Project + srsRAN 4G

- **Escopo:** gNB/eNB e UE open source; conformidade 3GPP; suporte O-RAN (ex.: split 7.2x RU).
- **Componentes no lab:** srsRAN Project gNB (5G SA) + srsRAN 4G srsUE (NR SA via ZMQ).
- **Deploy:** embedded, servidor ou cloud (K8s); rádio real (SDR) ou emulado (**ZMQ**).
- **Integração:** Open5GS/free5GC — N2 (AMF), N3 (UPF), mesmo PLMN/TAC/slice do subscriber.

O que é ZMQ neste lab?

ZeroMQ — transporte de amostras IQ em software

- **Problema:** testar PHY/MAC sem SDR — gNB e UE em processos/containers distintos.
- **Solução:** driver de rádio `device_driver: zmq` troca amostras IQ (como cabo de RF virtual).
- **Taxa:** `base_srate=23.04e6` (20 MHz NR típico) — tráfego significativo entre containers.
- **Não confundir:** ZMQ aqui é *front-end de rádio*, não mensageria A1/E2/O1.

Arquitetura do lab — qual projeto usar?

- **Projeto 1:** só UERANSIM monolítico — aprender Core.
- **Projeto 2:** UERANSIM split + srsRAN gNB monolítico + srsUE — PHY ZMQ sem F1 explícito.
- **Projeto 3:** mesmo Core; escolher `e2e` (split-8 interno) ou `strict-cudu` (F1 real entre containers).
- **Core comum:** AMF N2 `10.20.0.11` · UPF-A N3 `10.30.0.21` · pool UE `10.60.0.0/16`.

ZMQ — pareamento de portas (gNB ou DU ↔ UE)

Projeto 2: `gnb.yaml` · Projeto 3 strict: `du.yaml` + `ue.conf`

Entidade	TX (envia IQ)	RX (recebe IQ)
gNB / DU	<code>tcp://*:2000</code>	<code>tcp://srsran-ue:2001</code>
UE	<code>tcp://*:2001</code>	<code>tcp://srsran-gnb:2000</code>

- Parâmetros comuns: `id=ue` , `base_srate=23.04e6` , banda/cell alinhados (band 3, 20 MHz, PLMN 00101, TAC 7).
- TX do gNB = RX do UE e vice-versa — erro de porta = link de rádio morto.

O que roda de fato (vs UERANSIM)

- L1 (PHY): modulação/demodulação, FFT, precoding — amostras IQ via ZMQ.
- L2 (MAC/RLC/PDCP): scheduler, HARQ, bearers — visível em logs e métricas.
- L3 (RRC/NAS): registro 5G, PDU Session — interfaces N1 (NAS) e N2 (NGAP) reais.
- **Ainda ausente:** interface E2 — virá com OAI+FlexRIC (Aulas 04-06), como no NGO (proxy E2).

Ordem de subida e sincronização

Regras por projeto

- **Projeto 1:** `up_core.sh` → `up_ran.sh` (gNB+UE no mesmo container).
- **Projeto 2:** + `build-srsue.sh` · gNB ZMQ antes do srsUE (`up_ran.sh`).
- **Projeto 3 strict:** CU → DU → srsUE (`up_strict_cudu.sh`); UE ZMQ aponta para `srsran-du`.

Reiniciar só um lado do par ZMQ dessincroniza o link — reinicie gNB/DU+UE juntos.

Conectar com Polese, NGO e seminário §6.1

- **N2/N3 (3GPP):** já abertos no lab srsRAN — gNB multi-vendor com Core Open5GS.
- **O1/E2 (O-RAN):** NGO mede KPM em campus QCT; nós treinamos E2 com OAI+FlexRIC.
- **A1:** rApps NGO publicam políticas; xApps near-RT executam — lab ZMQ prepara entender *onde* KPM nasce (PHY/MAC).
- **Próximo passo:** trocar ZMQ por RFSIM/OAI e expor métricas via E2SM-KPM.

Discussão guiada — desagregação e ZMQ

Perguntas para a turma (12:00–12:45)

1. O que ganhamos ao separar gNB e UE em containers (Projeto 2) vs monólito (Projeto 1)?
2. Por que srsRAN+ZMQ é necessário antes de estudar KPM/E2?
3. O que trafega em `net-air` vs `net-zmq` vs `net-f1c` ?
4. Qual a diferença entre `gnb_split_8` (e2e) e `srsdu / srsdu` (strict-cudu)?
5. Como essa escada se compara ao testbed NGO (RAN comercial + proxy E2)?

Encaminhamento — Projeto 2 (Aula 04+)

- Stack: OAI CN + gNB + FlexRIC — `oai-cn-gnb/`
- Roteiros: a definir.
- De ZMQ (IQ) para **RFSIM** (OAI) + interface **E2**.

Material de apoio — Aula 03

- Módulo 5 — Aula 01/03 — srsRAN, UERANSIM; Interface A1, EI, A1 vs O1/E2.
- NGO et al., ICT Express 2024.
- srsRAN ZMQ lab — Projetos 2 e 3
- CU/DU estrito.
- Polese et al. — Non-RT RIC, A1, policy (IEEE ComST, 2023).
- O-RAN Alliance — **A1 interface** specifications.

Plataforma Web com exercícios propostos

Acesse os links abaixo para resolver os exercícios propostos:

 Atividades de Interfaces e Protocolos O-RAN →

 Atividades de RAN Intelligent Controller (RIC) →

Aula 04 — NGO §3: comercial vs open source + lab

- Juniper, VMware, Capgemini, Mavenir vs OSC/FlexRIC/SD-RAN
- Início do **Projeto 2** — lab OAI+FlexRIC (Aulas 04–06)



Referências

NGO, M. V. et al. "RAN Intelligent Controller (RIC)..." *ICT Express* 10 (2024).
(Fig. 1, 6–7; §3.1, §4, §6.1)

Polese, M. et al. "Understanding O-RAN." *IEEE ComST*, 2023. (A1, Non-RT
RIC — Módulo 5, Aula 03)



Referências complementares

O-RAN Alliance — A1 interface specifications

Dataset SUTD 5G — github.com/FCCLab/sutd_5g_dataset_2023

O-RAN SC NONRTRIC — o-ran-sc.org

srsRAN Project — docs.srsran.com · ZMQ app note 5G SA



Jonas Augusto Kunzler



Email: jak@cesar.school



LinkedIn: [jakunzler](https://www.linkedin.com/in/jakunzler)



GitHub: [jakunzler](https://github.com/jakunzler)



Instagram: [@jakunzler](https://www.instagram.com/jakunzler)